鸿蒙内核智能优化 · AGENT 长期记忆与质量治理基础设施

面向鸿蒙内核智能优化的团队级自进化经验中枢 Team Skill Hub

双资产双引擎治理 × 全自动闭环 —— 让团队优化经验像「私有 npm 包」一样可积累、可验证、可审计、可复利

从「一次性能力」到「可复利资产」
One-shot Capability → Compounding Assets

场景问题

规模化的真瓶颈：不是单次优化，而是经验复利

经验孤岛

哪个招式有效、哪里有坑、热函数怎么改、哪种验证不可信 —— 只存在工程师个人本地，无法跨人、跨项目、跨时间复用

重复踩坑 · 无法传承

团队反复探索同一方向、重蹈同一坑；专家经验带不走，新人与新 Agent 每次从零开始，优化边际效率不升反降

「简单共享」= 两大致命风险

① 知识漂移：经验自相矛盾、随时间失效；② 反馈自增强：均值变好、个别场景被悄悄做坏 —— 业界 mem0 / Zep 均缺团队级治理，规模越大风险越大

需要的不是又一个「记忆库」，而是带质量治理、可自进化的团队级经验基础设施

方案

版本化经验中枢 × 双资产双引擎 × 「消费 → 蒸馏 → 门控 → 发布 → 再消费」全自动闭环

hm-skill-hub · 团队中央经验仓 —— semver 发布 + lockfile 钉版 · 像「私有 npm」一样消费 · 可回滚 · 全程可审计

Knowledge 知识资产（事实 · 教训 · 坏招） 引擎 A · 治理型合并
只追加 + 七路关系分类（重复 / 矛盾 / 过时 / 条件 / 泛化 / 漂移 / 口径）· 双时态墓碑，永不物删

Skills 技能资产（做法 · 流程） 引擎 B · 竞争式进化
SkillOpt 有界编辑：留出 eval「严格变好且零回归」才接受 + Pareto 前沿防多人编辑塌缩

1 消费 · 自动挂载
流水线 research→...→test 按阶段注入 Hub 上下文；MCP 接入 · 6 个接入点 · 断连静默降级不阻塞

→

2 收口 · 自动蒸馏
run 收口双段抽取（确定性规则 + LLM 显著性）→ schema 合规 Tier-1 候选，自动附出处与证据

→

3 门控 · 策展入库
CI 五道门（lint · 脱敏 · 去重 · eval-gate）+ 双人评审；≥ 2 独立实例自动检测毕业为 technique 技能

过三道质量门 → 入库

4 发布回灌：nightly 七步作业 → semver + scorecard → broadcast 自动钉版

核心创新与差异化

01 双资产 · 双引擎
知识与技能严格分治、各配治理引擎，根除「漂移」与「坏经验覆盖」—— mem0 / Zep 缺失的团队治理层

02 七路分类 × 双时态
矛盾消解永不物删：superseded 墓碑 + 有效期，全程可审计；分类基准 48 例准确率 100%、误删 0

03 eval-gate 反喂安全
技能 = 可训练外部参数：改动须「严格变好且零回归」才入库；Pareto 留互补候选，根治自增强跑偏

04 全自动闭环工程化
收口蒸馏 → CI 门 → nightly 七步 → semver 发布 → 钉版回灌全自动；人只握晋升审批权

预期效果

经验从「个人消耗品」变成「团队复利资产」

开箱即会
新人 / 新 Agent 首次部署即携带全队经验，重复探索与重复踩坑显著减少

专家经验资产化
高手经验沉淀为质量门保护的团队资产，不再随人员流动而流失

复利式提升
经验跨成员、跨迭代持续累积，优化吞吐与命中率随使用不断上升

通用自进化底座
内存底噪 / 指令数 / 功耗 / 编译器后端等任意 Agent 优化场景即插即用

落地验证 全链路已实现、离线可复现：154 项测试 · 7 份 Schema · 5 道 CI 门 · 3 个 MCP 工具 · 6 个接入点 · 16 项机制词表 · 检索 recall@5=1.0（26 查询）· 七路分类 48/48 · 优化门 0.67→1.00

HARMONY KERNEL INTELLIGENT OPTIMIZATION · LONG-TERM MEMORY & QUALITY GOVERNANCE FOR AGENTS

Team-Level Self-Evolving Experience Hub Team Skill Hub

One-Shot Capability → Compounding Assets
Long-term memory infra for optimization agents

Dual engines × fully automated loop — team experience as a private npm package: accumulable · verifiable · auditable · compounding

Scenario & Pain

At scale the bottleneck is compounding experience, not one-shot wins

Experience silos

Effective moves, known pits, hot-function fixes, unreliable validations — all trapped in one engineer's workspace; never reused across people, projects, or time

Repeated pitfalls, no inheritance

Teams re-explore the same directions and re-step into the same traps; every new member or agent restarts from zero — marginal efficiency declines

Naive sharing = two fatal risks

① Knowledge drift — entries contradict and decay; ② self-reinforcing feedback — averages improve, edge cases silently regress; mem0 / Zep lack team-level governance

Needed: not another memory store — a governed, self-evolving team experience infrastructure

Solution

Versioned hub × dual engines × fully automated "consume → distill → gate → release → re-consume" loop

hm-skill-hub · central team hub — semver releases + lockfile pinning · a "private npm" for experience · rollbackable · auditable

Knowledge (facts · lessons · bad plans) Engine A · governed merge
Append-only + 7-way relation classes (dup / contradiction / temporal / conditional / subsumption / drift / basis) · bitemporal tombstones, never delete

Skills (procedures · playbooks) Engine B · competitive evolution
SkillOpt bounded edits — accepted only if strictly better with zero regression on held-out evals; Pareto frontier keeps complementary candidates

4 **Release & feed back: nightly 7-step job → semver + scorecard → broadcast re-pins — next run carries the whole team's experience**

3 quality gates → land

1 Consume · auto-mount
Staged pipeline (research→…→test) gets per-stage "Hub context" injection; MCP-integrated · 6 touchpoints · fail-silent degradation

→

2 Distill at close-out
Two-stage extraction (deterministic rules + LLM salience) → schema-valid Tier-1 candidates with provenance & evidence

→

3 Gate · curate
5 CI gates (lint · redact · dedup · eval-gate) + dual review; ≥2 independent instances auto-graduate into a technique skill

Core Innovations & Differentiation

01 Dual asset · dual engine
Knowledge and skills strictly separated, each with its own governance engine — the team-governance layer mem0 / Zep lack

02 7-way classes × bitemporal
Conflict resolution never deletes history: superseded tombstones + validity windows; 48-case benchmark: 100% accuracy, 0 false-deletes

03 Eval-gated feedback safety
Skill text = trainable external parameters: only strictly-better, zero-regression edits land; Pareto kills self-reinforcement

04 Fully automated loop
Close-out distill → CI gates → nightly 7 steps → semver release → re-pin, all automated; humans keep promotion approval

Expected Impact

Experience: from personal consumable to compounding team asset

Day-one mastery
New members & agents deploy with the whole team's experience — far less re-exploration

Experts → assets
Expert know-how becomes gate-protected team assets that survive turnover

Compounding gains
Throughput and hit-rate keep rising as experience accrues across members & iterations

Universal memory base
Plug-in for any agent-driven optimization: memory noise floor / instructions / power / compiler backends

Implemented End-to-end & reproducible offline: 154 tests · 7 schemas · 5 CI gates · 3 MCP tools · 6 touchpoints · 16-mechanism vocab · retrieval recall@5 = 1.0 (26 queries) · classifier 48/48 · skill-opt 0.67 → 1.00